

Práctica 2:

Análisis Estadístico y Generación Aleatoria

1 de junio de 2026

Unidad de Aprendizaje:	Programación en Ingeniería
Periodo:	Enero-Junio 2026
Profesor:	Dr. Mario Alberto Ibarra-Manzano
Fecha de entrega:	10 de junio de 2025
Medio de entrega:	Electrónico (Microsoft Teams)

Objetivo

Realice un programa que le permita obtener automáticamente el análisis estadístico de los datos ingresados en arreglos por el usuario o generados automáticamente dentro del rango definido $[min, max]$. El número máximo de muestras (N) será de 100,000.

Cuando los datos se generen automáticamente, el programa debe permitir seleccionar al menos **tres distribuciones de probabilidad**:

- **Uniforme** $U(min, max)$,
- **Normal (Gaussiana)** $\mathcal{N}(\mu, \sigma^2)$, y
- **Laplace (doble exponencial)** $Laplace(\mu, b)$ (útil en IA por su relación con el ruido, su robustez y su regularización ℓ_1).

Para garantizar que los datos estén dentro del rango $[min, max]$, se permite usar **recorte (clipping)** o **rechazo** (remuestreo) cuando un valor caiga fuera del intervalo. Incluya una opción para fijar la **semilla** del generador pseudoaleatorio y así asegurar la reproducibilidad.

Parámetros de las distribuciones: el programa debe permitir dos formas de configurar la generación aleatoria:

- Modo por rango** (mínimo y máximo): el usuario proporciona min y max .
- Modo por parámetros propios de la distribución:** por ejemplo, media μ y varianza σ^2 (o desviación estándar σ) para la normal; y μ y varianza (o parámetro b) para Laplace.

Ingreso de parámetros: el usuario podrá ingresar los parámetros **(a)** por consola usando `printf/scanf` o **(b)** por argumentos de línea de comandos (`argc, argv`). En el reporte, describe cuál modalidad implementaste (o ambas) y muestra un ejemplo de ejecución reproducible (incluyendo la semilla).

1. Medidas estadísticas

Suponiendo que el vector de muestras es:

$$X = \{x_1, x_2, x_3, x_4, \dots, x_i, \dots, x_N\} \quad (1)$$

donde la cardinalidad del vector, es decir el número de muestras en el vector, esta definida como:

$$n = \# \{X\} \quad (2)$$

El vector ordenado de forma ascendente de X esta definido como:

$$Y = \text{orden}(X) \quad (3)$$

El histograma del vector X esta definido como:

$$h = \{i \in [\min, \max] : h(i) = \# \{X = i\}\} \quad (4)$$

El programa debe poder obtener las siguientes quince medidas estadísticas:

- Máximo

$$\max = Y_n \quad (5)$$

- Mínimo

$$\min = Y_1 \quad (6)$$

- Media aritmética

$$\mu = \sum_{i=1}^n \frac{X_i}{n} \quad (7)$$

- Media geométrica

$$\sqrt[n]{\prod_{i=1}^n X_i} \quad (8)$$

- Mediana

$$\begin{cases} Y_{\frac{n+1}{2}} & \text{n es impar} \\ \frac{Y_{\frac{n}{2}} + Y_{\frac{n}{2}+1}}{2} & \text{n es par} \end{cases} \quad (9)$$

- Moda

$$X_{\max(h)} \quad (10)$$

- Varianza

$$\sigma^2 = \sum_{i=1}^n \frac{(x_i - \mu)^2}{n} \quad (11)$$

- Desviación estándar

$$\sigma = \sqrt{\sum_{i=1}^n \frac{(x_i - \mu)^2}{n}} \quad (12)$$

- Coeficiente de variación

$$CV = \frac{\sigma}{|\mu|} \quad (13)$$

- Cuartiles

$$Q_i = Y_{\frac{i \times n}{4}} \quad i = \{1, 2, 3, 4\} \quad (14)$$

- Deciles

$$D_i = Y_{\frac{i \times n}{10}} \quad i = \{1, 2, 3, \dots, 10\} \quad (15)$$

- Percentiles

$$P_i = Y_{\frac{i \times n}{100}} \quad i = \{1, 2, 3, \dots, 100\} \quad (16)$$

- Momentos no centrados de orden cero a cuatro orden.

$$\begin{array}{llll} \text{Orden 0} & \alpha_0 & = & 1 \\ \text{Orden 1} & \alpha = \mu & = & \sum_{i=1}^n \frac{X_i}{n} \\ \text{Orden 2} & \alpha_2 & = & \sum_{i=1}^n \frac{X_i^2}{n} \\ \text{Orden 3} & \alpha_3 & = & \sum_{i=1}^n \frac{X_i^3}{n} \\ \text{Orden 4} & \alpha_4 & = & \sum_{i=1}^n \frac{X_i^4}{n} \end{array} \quad (17)$$

- Momentos centrados de orden cero a cuatro orden

$$\begin{aligned}
\text{Orden 0 } \mu_0 &= 1 \\
\text{Orden 1 } \mu_1 &= 0 \\
\text{Orden 2 } \mu_2 &= \sum_{i=1}^n \frac{(X_i - \mu)^2}{n} = \alpha_2 - \alpha^2 = \sigma^2 \\
\text{Orden 3 } \mu_3 &= \sum_{i=1}^n \frac{(X_i - \mu)^3}{n} = \alpha_3 - 3\alpha\alpha_2 + 2\alpha^3 \\
\text{Orden 4 } \mu_4 &= \sum_{i=1}^n \frac{(X_i - \mu)^4}{n} = \alpha_4 - 4\alpha\alpha_3 + 6\alpha^2\alpha_2 - 3\alpha^4
\end{aligned} \tag{18}$$

- Momento estándar

$$\frac{\mu_k}{\sigma^k} \tag{19}$$

- Curtosis

$$b_2 = \frac{\sum_{i=1}^n \frac{(X_i - \mu)^4}{n}}{\sigma^4} \tag{20}$$

2. Consideraciones

- El programa prodrá capturar los datos necesarios mediante dos opciones: ingresar dato por dato o generar los n datos aleatorios en un rango de $[min, max]$.
- Los parámetros de las distribuciones pueden ingresarse en dos modos:
 - **Modo rango:** min y max .
 - **Modo parámetros:** p.ej. μ y σ^2 (o σ) para normal; μ y b (o varianza) para Laplace.
 - **Ordenamiento:** documenta qué medidas requieren ordenar (mediana, percentiles, cuartiles) y separa el tiempo de ordenamiento del tiempo de cálculo de medidas.
 - **Restricciones por tipo de dato:** *Counting sort* requiere valores enteros (o discretizados) con rango k razonable. *Bucket sort* requiere definir el número de casilleros y el criterio de asignación; su desempeño depende de la distribución.

En ambos casos, el reporte debe indicar claramente qué parámetros se usaron.

- El ingreso de parámetros se podrá realizar por (a) consola con `printf/scanf` o (b) argumentos `argc/argv`. Para valor extra, implementa ambas modalidades.
- La generación automática debe permitir al menos **tres distribuciones: uniforme, normal y Laplace**. Para normal y Laplace se recomienda parametrizar usando el intervalo $[min, max]$:
 - Normal: $\mu = \frac{min+max}{2}$ y $\sigma = \frac{max-min}{6}$ (aprox. $\pm 3\sigma$ cubre el rango).
 - Laplace: $\mu = \frac{min+max}{2}$ y $b = \frac{max-min}{6\sqrt{2}}$ (ya que $std = \sqrt{2}b$).

Si un valor generado cae fuera de $[min, max]$, aplique recorte o re-muestreo y documente cuál opción usó.

- Todas las medidas estadísticas deben estar almacenadas en uno o varios arreglos y exhibir los resultados en una tabla.
- Todas la medidas deben estar en un solo programa y cada una de ellas debe ser una sola función separada, esta función puede llamar otras funciones, pero una función no puede calcular dos medidas.
- Realice la simulación para 20 casos diferentes según la siguiente tabla:
- En el reporte se debe incluir una tabla donde la primer columna sea el **caso**, la segunda la **distribución** (Uniforme/Normal/Laplace), la tercera el número n de datos y las siguientes sean las diversas medidas y sus valores. La última columna debe reportar el **tiempo** requerido para calcular las medidas (se recomienda medir solo el cálculo, no la generación).
- Optimice los algoritmos de ordenamiento y cálculo de medidas para reducir la complejidad en el programa.
- Acelere el funcionamiento de los algoritmos utilizando la recurrencia en las funciones que lo permita.

#	n	Máximo	Mínimo	#	n	Máximo	Mínimo
1	100	-1	1	11	4000	1	-1
2	1000	5	-5	12	5000	1	-1
3	1000	20	-10	13	6000	1	-1
4	1000	0	-50	14	7000	1	-1
5	1000	1	-1	15	8000	1	-1
6	2000	30	10	16	9000	1	-1
7	2000	30	-30	17	10000	1	-1
8	2000	-10	-50	18	20000	1	-1
9	2000	150	-100	19	30000	1	-1
10	2000	1	-1	20	50000	1	-1

Cuadro 1: Casos de simulación para generación de datos (20 escenarios).

3. Generación de números aleatorios (distribuciones)

Cuando selecciones la opción de generación automática, el programa deberá generar el vector de muestras $X = \{x_1, \dots, x_n\}$ con una distribución seleccionada por el usuario.

Conversión base: `rand()` a uniforme continua

La función `rand()` genera enteros en $[0, \text{RAND_MAX}]$. Para obtener una uniforme continua se recomienda:

$$u = \frac{\text{rand()} + 1}{\text{RAND_MAX} + 2} \in (0, 1), \quad (21)$$

lo cual evita $u = 0$ y $u = 1$ (importante cuando se usa $\ln(u)$).

Dos modos de parametrización (obligatorio)

El programa debe permitir dos formas de ingresar parámetros:

- Modo 1: por rango** (mínimo y máximo). El usuario ingresa *min* y *max*.
- Modo 2: por parámetros propios de la distribución**. Ejemplos:
 - Normal: media μ y varianza σ^2 (o desviación estándar σ).
 - Laplace: ubicación μ y escala b (o varianza).

Distribución uniforme $U(\min, \max)$

Modo rango. Con $u \sim U(0, 1)$:

$$x = \min + (\max - \min)u \sim U(\min, \max). \quad (22)$$

Modo parámetros. En uniforme es natural usar el rango; si el usuario proporciona media m y varianza v , entonces:

$$(\text{equivalente}) \quad \min = m - \sqrt{3v}, \quad \max = m + \sqrt{3v}. \quad (23)$$

Distribución normal $\mathcal{N}(\mu, \sigma^2)$ (transformación de Box–Muller)

A partir de $u_1, u_2 \sim U(0, 1)$ se construye:

$$z = \sqrt{-2 \ln(u_1)} \cos(2\pi u_2), \quad x = \mu + \sigma z. \quad (24)$$

Modo parámetros. Si se ingresa varianza σ^2 , entonces $\sigma = \sqrt{\sigma^2}$.

Modo rango (parámetros derivados). Una parametrización práctica (aprox. $\pm 3\sigma$ dentro del rango) es:

$$\mu = \frac{\min + \max}{2}, \quad \sigma = \frac{\max - \min}{6}. \quad (25)$$

Distribución Laplace (doble exponencial) $\text{Laplace}(\mu, b)$ (transformación inversa)

Con $u \sim U(0, 1)$:

$$x = \mu - b \operatorname{sgn}(u - 0.5) \ln(1 - 2|u - 0.5|). \quad (26)$$

Modo parámetros. Si se ingresa varianza v , se usa la relación $v = 2b^2$, es decir:

$$b = \sqrt{\frac{v}{2}}. \quad (27)$$

Modo rango (parámetros derivados). Usando el criterio de $\pm 3\sigma$ y $\sigma = \sqrt{2}b$:

$$\mu = \frac{\min + \max}{2}, \quad b = \frac{\max - \min}{6\sqrt{2}}. \quad (28)$$

Restricción al intervalo $[\min, \max]$

Si la muestra generada cae fuera de $[\min, \max]$, aplica una de las dos estrategias (y documenta cuál usaste):

- **Recorte (clipping):** $x \leftarrow \min(\max(x, \min), \max)$.
- **Rechazo (re-muestreo):** repetir el muestreo hasta que $x \in [\min, \max]$.

En **modo parámetros**, si el usuario no define $\min$ y $\max$, puedes proponer automáticamente un rango (p. ej. $[\mu - 3\sigma, \mu + 3\sigma]$ para normal; $[\mu - 3\sqrt{2}b, \mu + 3\sqrt{2}b]$ para Laplace) y reportarlo.

Ingreso de parámetros (obligatorio)

El programa deberá permitir configurar la simulación mediante:

- **Consola (interactivo):** usando `printf` y `scanf`.
- **Línea de comandos:** usando `argc/argv` (por ejemplo, para ejecución reproducible en lotes).

Ejemplos de uso (sugeridos):

```
# Modo por rango
./practica2 --dist normal --mode range --min -1 --max 1 --n 2000 --seed 123

# Modo por parámetros (mu,var)
./practica2 --dist normal --mode params --mu 0 --var 0.04 --n 2000 --seed 123
./practica2 --dist laplace --mode params --mu 0 --var 0.5 --n 2000 --seed 123
```

Reproducibilidad

Incluye una opción de semilla (`srand(seed)`). En el reporte indica el valor de semilla usado para reproducir resultados.

4. Ordenamiento: algoritmos, complejidad y efecto de la distribución

Varias medidas estadísticas (por ejemplo: mediana, cuartiles y percentiles) requieren **ordenar** el conjunto de datos. En esta práctica, además de calcular las medidas, deberás **implementar y comparar** diferentes algoritmos de ordenamiento, analizando cómo cambia su desempeño según la **distribución** y las propiedades del conjunto (rango, repetición de valores, sesgo, etc.).

Algoritmos a implementar (obligatorio)

Implementa cada algoritmo como una función independiente (por ejemplo, `void bubble(double *a, int n)`), y agrega una opción para seleccionar el método:

- Algoritmo burbuja (*Bubble sort*)
- Algoritmo de pares y nones (*Odd-Even sort*)
- Algoritmo de selección (*Selection sort*)
- Algoritmo de inserción (*Insertion sort*)

- Algoritmo de casilleros (*Bucket sort*)
- Algoritmo burbuja bidireccional (*Cocktail sort*)
- Algoritmo Shell (*Shell sort*)
- Algoritmo de conteo (*Counting sort*)

Complejidad esperada (referencia para tu análisis)

- Bubble / Odd–Even / Selection / Insertion / Cocktail: tiempo típico $\mathcal{O}(n^2)$, memoria $\mathcal{O}(1)$.
- Shell: depende de la secuencia de incrementos; típicamente mejor que $\mathcal{O}(n^2)$ y cercano a $\mathcal{O}(n^{3/2})$ en práctica; memoria $\mathcal{O}(1)$.
- Bucket: tiempo esperado $\mathcal{O}(n + k)$ (si la distribución reparte bien los elementos); memoria $\mathcal{O}(n + k)$.
- Counting: tiempo $\mathcal{O}(n + k)$; memoria $\mathcal{O}(k)$, donde k es el tamaño del rango discreto.

Efecto de la distribución y selección del algoritmo (lo que debes discutir)

En el reporte, compara el tiempo total de ordenamiento para los 20 casos y para las tres distribuciones (Uniforme, Normal, Laplace), y discute qué algoritmos resultan más convenientes y por qué. Considera al menos:

- **Uniforme (rango conocido):** tiende a dispersar valores de forma homogénea, lo cual favorece *Bucket sort* si los casilleros están bien elegidos.
- **Normal (concentración en torno a μ):** puede generar alta concentración en un subrango; en *Bucket sort* esto puede producir casilleros desbalanceados (peor desempeño), a menos que adaptes el número de casilleros o uses una estrategia de asignación adecuada.
- **Laplace (colas pesadas):** aumenta la probabilidad de valores extremos; puede afectar la elección de *rango* y el comportamiento de casilleros.
- **Datos casi ordenados:** Insertion sort suele mejorar mucho (cercano a $\mathcal{O}(n)$) cuando el arreglo ya está parcialmente ordenado.
- **Counting sort (caso especial):** es apropiado cuando los datos son **enteros** o pueden mapearse a enteros con un rango k razonable. Si tu conjunto es de tipo **double**, debes explicar el mapeo (por ejemplo, discretización por bins) y el impacto en precisión.

Requisito de medición

Mide el tiempo de ordenamiento por algoritmo (**por separado del cálculo de medidas**) y repórtalo en la tabla final. Además, indica cuál algoritmo seleccionas como **mejor** para cada distribución (o para cada caso) y justifica tu decisión.

Rúbrica de evaluación (total: 100 puntos)

Entregables en Teams

Sube al canal correspondiente en Microsoft Teams:

- Código fuente en C** (carpeta o repositorio) con:
 - validación de entradas y menú/flujo de captura o generación automática,
 - funciones separadas por medida estadística (una función por medida),
 - tabla de resultados por caso (incluyendo el tiempo total de cómputo).
- Reporte PDF** (2–6 páginas) con:
 - descripción del diseño del programa (módulos y funciones),
 - tabla(s) de resultados de los 20 casos (véase Tabla 1),
 - discusión de complejidad temporal y espacial (principales cuellos de botella),
 - evidencias (capturas de ejecución) y conclusiones.

Documentación con Doxygen (recomendado)

Incluye cabeceras Doxygen en cada función (por ejemplo: `@brief`, `@param`, `@return`) y genera documentación HTML. Si usas Graphviz (`HAVE_DOT=YES`), podrás obtener diagramas de llamadas (call graphs) útiles para evi-

Criterio	Pts.	Excelente	Muy bueno	Satisfactorio	Básico	Insuficiente
1) Algoritmo funcional y flujo de ejecución (menú, I/O, validación; consola y/o argv)	15	Compila/ejecuta; flujo claro; siempre; validación robusta; manejo de errores; consola y/o argv correctamente implementados y documentados.	Funciona casi siempre; validación adecuada con detalles menores.	Funciona en casos típicos; validación parcial; interfaz limitada.	Fallas frecuentes o flujo confuso; ejecución mínima.	No compila, no ejecuta o no corresponde al problema.
2) Medidas estadísticas (15 medidas)	20	Todas correctas y verificables; coincide con definiciones; casos límite atendidos; resultados consistentes.	Casi todas correctas; 1-2 discrepancias menores.	Cumple lo esencial pero hay varias imprecisiones.	Varias medidas incorrectas o incompletas.	No implementa o implementa incorrectamente la mayoría.
3) Generación aleatoria (Uniforme, Normal, Laplace) con dos modos de parámetros + semilla	15	Incluye 3 distribuciones; soporta modo rango y modo parámetros; transformaciones correctas; semilla configurable; estrategia fuera de rango documentada.	Incluye 3 distribuciones y ambos modos con detalles menores.	Incluye 3 distribuciones pero solo 1 modo o rango mal aplicado en algunos casos.	Incluye 1-2 distribuciones o implementación superficial.	No incluye generación por distribuciones o es incorrecta.
4) Ordenamiento: implementación de 8 algoritmos + selección justificada según distribución	25	Implementa los 8 algoritmos solicitados; mide tiempos por algoritmo; compara por distribución; elige el mejor (por caso o por distribución) y lo justifica con evidencia.	Implementa la mayoría y compara razonablemente; justificación breve.	Implementa varios (4-5) y reporta algunos tiempos; comparación limitada.	Implementa pocos (1-3) y medición clara.	No implementa ordenamientos o son incorrectos.
5) Simulación de 20 casos + tablas (caso, distribución, n , medidas, tiempos)	15	Resultados completos para los 20 casos y 3 distribuciones; tablas claras; separa tiempo de ordenamiento y de cálculo.	Casi completo; faltan pocos o alguna distribución.	Tablas presentes pero incompletas o poco claras.	Evidencia limitada; faltan muchos casos.	Sin evidencia verificable o tablas ausentes.
6) Reporte y documentación (PDF + Doxygen)	10	Reporte estructurado (método, resultados, discusión); capturas; Doxygen completo por función; HTML generado (y DOT si aplica).	Reporte y Doxygen completos con detalles menores.	Reporte entendible; Doxygen mínimo.	Reporte pobre/-desordenado; documentación escasa.	No hay reporte o documentación.

Cuadro 2: Rúbrica de evaluación para la Práctica 2.

denciar la modularización del programa.